

Esta tarea tiene como objetivo diseñar una gramática para un analizador LR e implementar una aplicación capaz de procesar archivos de letras de canciones sincronizadas (.lrc) utilizados en conjunto con un reproductor básico de archivos MP3.

1) Enunciado

Actualmente, los reproductores de música han evolucionado incorporando diversas funcionalidades orientadas a mejorar la experiencia del usuario. Una de las más utilizadas es la visualización sincronizada de letras de canciones mientras el audio se reproduce. Estas letras se almacenan generalmente en archivos de texto plano con extensión .lrc, los cuales contienen marcas de tiempo que indican el instante exacto en que cada línea de la canción debe aparecer en pantalla.

En esta tarea, deberá implementar un analizador sintáctico capaz de reconocer y procesar archivos .lrc, almacenando su contenido en memoria y sincronizando las letras con un reproductor básico de archivos MP3 implementado en Java. El reproductor base se encuentra implementado en la clase Reproductor.java, la cual utiliza la librería BasicPlayer para reproducir archivos MP3.

1.1) El formato de archivos Lyrics (.lrc)

Los archivos .lrc contienen texto sincronizado con una canción mediante etiquetas de tiempo. A continuación, se presenta un ejemplo:

```
[01:00.32]So close, no matter how far  
[01:05.41]Couldn't be much more from the heart  
[01:10.66]Forever trusting who we are  
[01:15.26]And nothing else matters
```

Cada línea del archivo contiene:

- Una marca de tiempo entre corchetes.
- El texto de la canción asociado a ese instante.

El formato del tiempo corresponde a: [mm:ss.xx]

Donde:

- mm representa los minutos.
- ss representa los segundos.
- xx representa centésimas de segundo.

Por ejemplo: [01:35.20]

indica que la línea debe mostrarse al minuto 1, segundo 35 y 20 centésimas.

1.2) Metadatos del archivo .lrc

Algunos archivos .lrc pueden incluir información adicional sobre la canción, típicamente esta información viene ubicada al principio del archivo:

```
[ar:Queen]
[ti:Bohemian Rhapsody]
[al:A Night at the Opera]
```

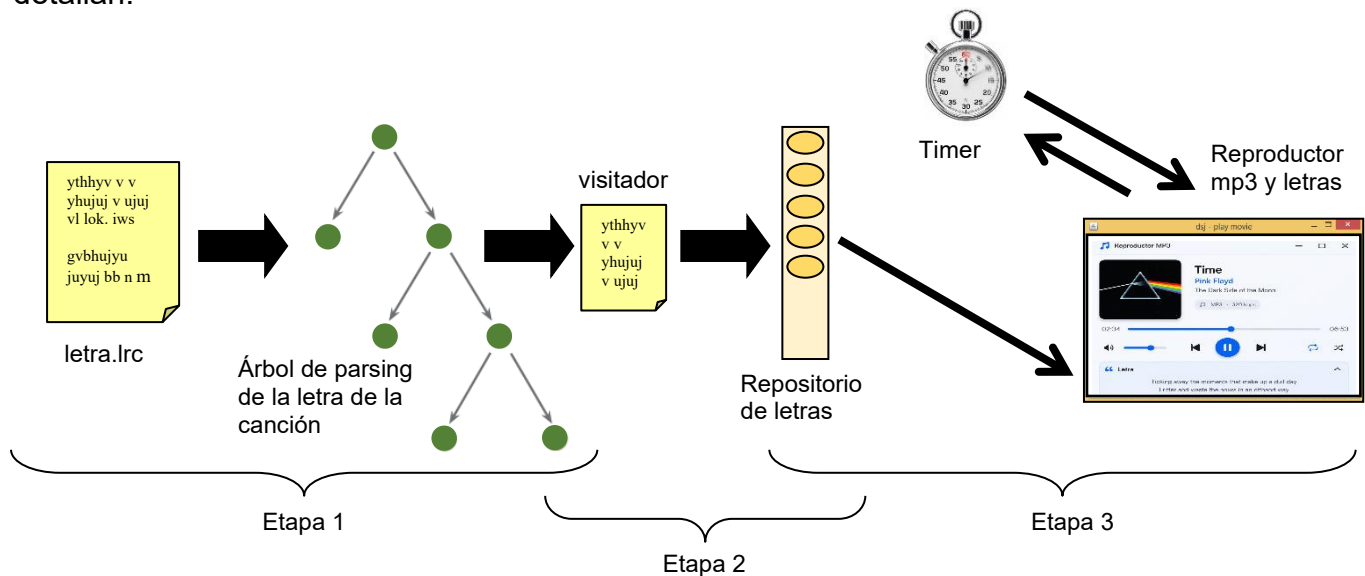
Donde:

- ar → artista
- ti → título de la canción
- al → álbum

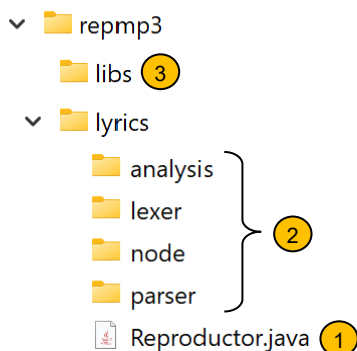
Estos datos también deben ser reconocidos por la gramática. Si hay otros metadatos adicionales a estos queda a su elección reconocerlos o no.

2) Requerimientos

Para hacer más fácil su trabajo, la tarea se ha dividido en tres etapas que a continuación se detallan:



Para trabajar en su tarea, se sugiere la siguiente estructura de carpetas:



1. Reproductor.java es el programa reproductor de mp3.
2. Carpetas generadas automáticamente por SableCC.
3. Librería para java que permite reproducir los mp3

2.1) Etapa 1: Diseño de la gramática

Para el reconocimiento y carga de los archivos con las letras de las canciones se debe utilizar un proceso de parsing Bottom-Up (LR), en este caso utilizando la herramienta SableCC <https://sablecc.org/>.

La gramática debe reconocer:

- Marcas de tiempo.
- Texto sincronizado.
- Metadatos del archivo.
- Múltiples líneas de lyrics.

Pasos sugeridos:

1. Modificar la gramática para aceptar el formato .lrc.
2. Ejecutar SableCC para generar el parser.
3. Modificar el programa Java para leer archivos .lrc.
4. Construir el árbol de derivación.

2.2) Etapa 2: implementación del visitador (reconocer el formato .lrc)

Una vez verificado el correcto reconocimiento de las letras de la canción por la gramática se debe implementar el código para almacenar en memoria principal cada subtítulo, pensando en su posterior recuperación.

Implemente un archivo nuevo para el visitador. Abra el archivo [DepthFirstAdapter.java](#) ubicado en la carpeta [analysis](#) que se generó automáticamente en el paso anterior. En este archivo, reconozca los métodos que visitan los nodos. Se aconseja realizar esta etapa visualizando el árbol de derivación mediante la utilización de [ASTDisplay.java](#), use también un lyric pequeño para que el árbol no crezca demasiado y se comprenda la estructura generada. Una vez identificados los métodos que debe modificar cópielos al visitador que usted programará, el resto de los métodos de [DepthFirstAdapter.java](#) no se copian ni modifican (pues se hereda de esta clase).

La labor principal del visitador será obtener la información de cada ítem del lyric y almacenarlo en un repositorio, por ejemplo, puede ser un [ArrayList](#). Cada elemento a almacenar, puede ser un objeto que usted defina, debe contener el tiempo y el texto del lyric. Una vez que el visitador termine de recorrer el árbol el [ArrayList](#) contendrá todos los lyrics de la canción.

2.3) Etapa 3: Integración al reproductor

En esta etapa se debe modificar el código del reproductor, [Reproductor.java](#). La versión del reproductor reproduce archivos .mp3 ya implementa un timer que permite programar una alarma que ejecute automáticamente un método cuando se cumpla el tiempo. Estudie con atención la clase [Reminder](#), ubicada al final del programa. Puede modificar el constructor para que acepte los parámetros que usted quiera utilizar. La idea general consiste en programar el timer para que avise cuando se cumple el tiempo para mostrar el subtítulo, en ese momento buscar el texto de la letra que se encuentra en el repositorio y desplegarlo con el formato deseado sobre la ventana del reproductor.

3) Archivos proveídos

En plataforma Moodle de la asignatura encontrará lo siguiente:

1. Librerías necesarias para reproducir un archivo mp3 (archivos .jar).
2. Dos canciones y lyrics de ejemplo.
3. Código base para el reproductor de mp3 y timer (.java)

4) Consideraciones de trabajo y entrega

- Grupos de 3 estudiantes máximo.
- Indique los integrantes del grupo como comentario al principio del programa java.
- Debe entregar su tarea en Moodle como un enlace google drive, el cual debe contener:
 - Un informe en pdf con la explicación de la gramática.
 - Archivo de especificación de la gramática de SableCC.
 - Código fuente completo del proyecto
 - Código compilado y funcionando del proyecto
 - 3 canciones mp3 y sus lyrics con los cuales probó su solución
 - proyecto
- La tarea también se probará con ejemplos propios de los profesores, por cuanto pruébalo con varios ejemplos diferentes.
- Fecha de entrega: viernes 12 de junio 2026 hasta las 23:59 hrs. Se descontará un punto por día de atraso.

Los profesores se reservan el derecho de solicitar presentación de la tarea de manera presencial.